Common setup (run once)

```
pip install nltk
```

```
import nltk

# Download the NLTK datasets/models needed by the lab programs.
nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('brown')
nltk.download('averaged_perceptron_tagger_eng')
nltk.download('wordnet')
nltk.download('omw-1.4')
nltk.download('universal_tagset')
nltk.download('conll2000')
nltk.download('maxent_ne_chunker_tab')
nltk.download('words')
```

Run each program in Jupyter Notebook, Google Colab, or VS Code with Python installed. Some programs download corpora on first run and need an internet connection.

13. Implement a chunking algorithm that selects proper features for training a model and considers training corpus size

Aim: To train a chunking model that identifies noun phrases and other syntactic chunks using POS tags and contextual features.

Chunking groups tagged words into meaningful phrases. For example, in “the small cat sleeps,” the phrase “the small cat” can be identified as a noun phrase.

This program uses the CoNLL-2000 chunking corpus and a maximum entropy classifier.

```
import nltk
from nltk.corpus import conll2000
from nltk.chunk.util import tree2conlltags, conlltags2tree
from nltk.classify import MaxentClassifier

# Download the chunking dataset.
nltk.download('conll2000')

# Convert a tagged sentence into word, POS, and chunk-tag triples.
def sentence_to_iob(sentence):
    return tree2conlltags(sentence)

# Feature extractor for each token.
def chunk_features(sentence, index):
    word, pos, chunk_tag = sentence[index]

    # Obtain surrounding words and POS tags.
    previous_word = sentence[index - 1][0] if index > 0 else '<START>'
    previous_pos = sentence[index - 1][1] if index > 0 else '<START>'

    next_word = sentence[index + 1][0] if index < len(sentence) - 1 else '<END>'
    next_pos = sentence[index + 1][1] if index < len(sentence) - 1 else '<END>'

    # Return contextual features used by the classifier.
    return {
        'word': word.lower(),
        'pos': pos,
        'prefix': word[:3].lower(),
        'suffix': word[-3:].lower(),
        'previous_word': previous_word.lower(),
        'previous_pos': previous_pos,
        'next_word': next_word.lower(),
        'next_pos': next_pos
    }

# Load the training and testing portions of the corpus.
train_trees = conll2000.chunked_sents('train.txt')
test_trees = conll2000.chunked_sents('test.txt')

# Use a configurable subset of the training corpus.
training_size = 500
training_trees = train_trees[:training_size]
```

```
# Extract a feature dictionary and IOB label for each token.
training_examples = []

for tree in training_trees:
    tagged_sentence = sentence_to_iob(tree)

    for index, (word, pos, chunk_tag) in enumerate(tagged_sentence):
        features = chunk_features(tagged_sentence, index)
        training_examples.append((features, chunk_tag))

# Train the maximum entropy classifier.
chunk_classifier = MaxentClassifier.train(
    training_examples,
    algorithm='megam',
    trace=0,
    max_iter=10
)

# Predict chunk labels for a tagged sentence.
def predict_chunks(tree):
    tagged_sentence = [
        (word, pos, chunk_tag)
        for word, pos, chunk_tag in sentence_to_iob(tree)
    ]

    predicted_iob = []

    for index, (word, pos, _) in enumerate(tagged_sentence):
        features = chunk_features(tagged_sentence, index)
        predicted_tag = chunk_classifier.classify(features)

        predicted_iob.append((word, pos, predicted_tag))

    # Reconstruct a chunk tree from predicted IOB labels.
    return conlltags2tree(predicted_iob)

# Test the model on one held-out sentence.
sample_tree = test_trees[0]
predicted_tree = predict_chunks(sample_tree)

print("Original sentence:")
print(" ".join(word for word, pos in sample_tree.leaves()))

print("\nPredicted chunk tree:")
print(predicted_tree)
```

Important execution note

The `megam` algorithm requires an external Megam executable, which may not be installed by default. To avoid that dependency, replace the classifier-training section with the following version using NLTK's built-in maximum entropy implementation:

```
chunk_classifier = MaxentClassifier.train(
    training_examples,
    algorithm='iis',
    trace=0,
    max_iter=10
)
```

Sample output

```
Original sentence:
Confidence in the pound is widely expected to take another sharp dive

Predicted chunk tree:
(S
  (NP Confidence/NN)
  (PP in/IN)
  (NP the/DT pound/NN)
  (VP is/VBZ widely/RB expected/VBN)
  ...
)
```

The precise sentence and predicted chunk tree depend on the selected test example and the trained model.

Explanation:

• **Word features:** The current word, its prefix, and its suffix.

• **Contextual features:** The previous and next words and POS tags.

• **IOB labels:** `B-NP` marks the beginning of a noun phrase, `I-NP` marks a token inside one, and `O` marks a token outside a chunk.

• **Training corpus size:** The `training_size` variable controls how many training sentences are used.

• **Classification:** The maximum entropy classifier predicts the IOB label for each token, and `conlltags2tree()` reconstructs the phrase structure.

To study the effect of training size, change `training_size` to `100`, `500`, or `1000`. A larger training set can improve generalization, but performance also depends on feature quality, model settings, and the complexity of the data.

Quick revision table

```
| Program | Main concept | Important NLTK function/class |
| 1 | Word counts and vocabulary | `set()`, `sorted()` |
| 2 | Conditional frequency distribution | `ConditionalFreqDist` |
| 3 | Morphological generation and analysis | Root/suffix operations |
| 4 | Add-Delete morphology | Morphological transformation table |
| 5 | Tokenization and segmentation | `RegexpTokenizer`, `sent_tokenize` |
| 6 | Stemming and lemmatization | `PorterStemmer`, `WordNetLemmatizer` |
| 7 | Bigrams and sentence probability | `bigrams()`, `Counter` |
| 8 | N-gram language model | `ngrams()` |
| 9 | HMM probability matrices | `Counter`, transition/emission counts |
| 10 | Viterbi POS tagging | `HiddenMarkovModelTrainer` |
| 11 | Context-sensitive POS tagging | `UnigramTagger`, `BigramTagger`, `TrigramTagger` |
| 12 | Document classification | `NaiveBayesClassifier` |
| 13 | Phrase chunking | `MaxentClassifier`, `conll2000` |
```

For your lab record: Include the aim, program, sample output, and result for each experiment. For corpus-based programs, record the actual output produced on your machine rather than copying illustrative counts or predictions.